Practicum Photography: Code Guide

Oct 17, 2026 · @Royne

Overview

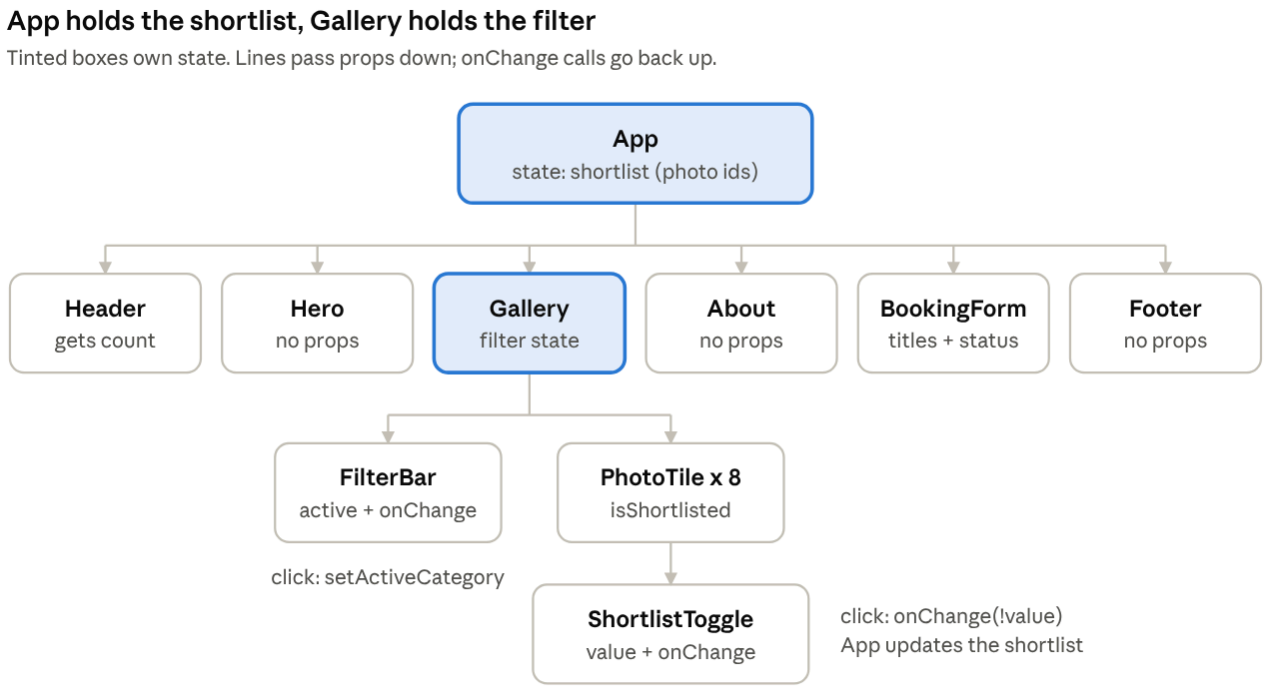
The site is one HTML page that React fills in. The browser loads `index.html`, which loads `src/main.jsx`, which draws `<App />` into the empty `<div id="root">`. Everything you see after that is built by components in `src/components/`.

Three tools do the work:

- **Vite** runs the dev server (`npm run dev`) and bundles the site for production (`npm run build`).
- **React** builds the page from components, small JavaScript functions that return markup (JSX).
- **Tailwind CSS v4** styles it with utility classes written straight into the markup, such as `px-4` or `sm:grid-cols-2`. There is no separate stylesheet per component; `src/index.css` only holds the colour and font tokens.

How a page load flows:

1. `index.html` gives the browser the fonts, the favicon and an empty `<div id="root">`.
2. `src/main.jsx` imports `index.css` and tells React to render `<App />` into that div.
3. `App.jsx` holds the shortlist state and lays out `Header`, `<main>` (Hero, Gallery, About, BookingForm) and `Footer`.
4. `Gallery.jsx` reads the photo list from `src/data/photos.js` and draws one `PhotoTile` per photo.
5. When you click a filter or a shortlist button, React updates the state and redraws only what changed.



component tree · where state lives

Only `App` and `Gallery` hold the page's main state. BookingForm keeps one small `status` message of its own; every other component just shows the props it is given.

HTML

The page's markup lives in two places: a short `index.html` shell, and JSX inside each component. JSX looks like HTML but is written in JavaScript files, so it can use variables and loops.

index.html: the shell

`Photography Portfolio/index.html` is almost empty on purpose. It holds:

- `<html lang="en">` so screen readers use English pronunciation, plus `scroll-smooth` so anchor links glide.
- The viewport `<meta>`, which makes phones use their real width instead of zooming out.
- `<title>`, the meta description, the `favicon.svg` link and the Google Fonts link (Instrument Serif, Manrope, IBM Plex Mono).
- `<body>` with the page colours (`bg-paper text-ink`), one `<div id="root"></div>` and the script tag that loads `src/main.jsx` .

JSX: HTML written inside components

Each component returns markup. Four differences from plain HTML matter in this project:

Plain HTML	JSX	Why
`class="..."`	`className="..."`	`class` is a reserved word in JavaScript
`for="email"` on a label	`htmlFor="email"`	`for` is reserved too (see `FormField.jsx`)
text typed by hand	`{photo.title}`	curly braces insert a JavaScript value
copy a block 8 times	`photos.map(...)`	a loop makes one element per item, each with a `key`

Semantic structure

The page uses elements that say what each part is, which helps screen readers and search engines:

- `Header.jsx`: a skip link ("Skip to content", hidden until you press Tab), then `<header>` with the brand and a `<nav aria-label="Main">` list.
- `App.jsx`: one `<main id="main">` around Hero, Gallery, About and BookingForm.
- Each section is a `<section>` with an `id` (`#top`, `#work`, `#about`, `#book`) the nav links jump to, and `aria-labelledby` pointing at its heading.
- `Hero.jsx` holds the page's only `<h1>`. Every other section has an `<h2>`; the gallery's is visually hidden with `sr-only` but still read aloud.
- Each photo is an `<li>` in a `<ul>`, holding a `<figure>` with an `<img>` and a `<figcaption>`.
- `About.jsx` uses a `<dl>` (description list) for "Based in / Travels to / Shooting since".
- `Footer.jsx`: `<footer>` with the copyright and a Back to top link.

Accessibility details

- Every `<img>` has `alt` text from the data file, plus `width` and `height` so the page doesn't jump as images load.
- The filter and shortlist buttons use `aria-pressed`, so a screen reader says "pressed" or "not pressed".
- The shortlist button sits **beside** the photo link, not inside it, because a `<button>` inside an `<a>` is invalid HTML.

- "Showing n of 8" and the form's status message use `aria-live="polite"`, so changes are announced.
- Every form control has a `<label>` linked by `htmlFor` / `id`, and required fields use `required`.
- A visible focus ring for keyboard users is set once in `index.css` with `:focus-visible`.

Styling with Tailwind classes

Styles are classes on the elements themselves. A few patterns to recognise:

- **Tokens:** `bg-paper`, `text-ink`, `text-accent`, `font-display` come from the `@theme` block in `src/index.css`. Change a colour there and every element using it follows. Dark mode swaps the same tokens.
- **Spacing and size:** `px-4` = padding left and right, `gap-6` = space between flex/grid children, `max-w-7xl` = maximum width.
- **Layout:** `flex`, `flex-col`, `grid`, `grid-cols-2`, `col-span-2`, `row-span-2`.
- **States:** `hover:`, `focus:`, `active:`, `group-hover:` (a tile reacts when its parent is hovered), `motion-reduce:` (no animation for people who turned it off).

Mobile-first breakpoints

A class with no prefix applies to every screen. A prefixed class switches on from that width up, so the phone layout is written first and wider layouts are layered on top.

Prefix	From	Example in the code
none	0px (phones)	`grid-cols-1` in `Gallery.jsx`
`sm:`	640px	`sm:grid-cols-2`, form fields `sm:flex-row`
`md:`	768px	About photo beside text, `md:flex-row`
`lg:`	1024px	`lg:grid-cols-4`, hero and booking side by side

Wide and feature tiles only get `sm:col-span-2`. On a phone's single column, a two-column span would push the page wider than the screen.

JavaScript

All the behaviour is plain JavaScript in React components: two main pieces of state, a data file and one helper. There is no TypeScript and no library beyond React and Tailwind.

Modules: import and export

Each file is a module. It shares things with `export` and uses other files' things with `import`:

```
// photos.js
export const categories = [ ... ];  // named export
export default photos;               // default export

// Gallery.jsx
import photos, { categories } from "../data/photos.js";
```

Every component file ends with `export default ComponentName;` so `App.jsx` can import it.

main.jsx: starting React

```
createRoot(document.getElementById("root")).render(
  <StrictMode><App /></StrictMode>
);
```

This finds the empty div from `index.html` and draws `App` inside it. `StrictMode` runs extra checks while developing and does nothing in the built site. The file also imports `index.css`, which is how Tailwind gets onto the page.

The data file: src/data/photos.js

The gallery content is one array of 8 objects. Each object describes one photo:

Field	Example	Used for
`id`	`1`	the React `key` and the shortlist
`title`	`"Savanna at sunset"`	caption and shortlist names
`category`	`"plains"`	which filter shows it
`alt`	`"Sunset over a dry, grassy savanna"`	the image's alt text
`shape`	`"feature"`	tile size: `single`, `tall`, `wide`, `feature`
`unsplashId`	`"photo-1761..."`	builds the image URLs
`credit`, `link`	photographer, Unsplash page	caption credit and tile link

The same file exports `categories`, the list of filter buttons. `"all"` isn't a real category; it means "no filter". Adding a photo means adding one object here, with no markup to copy.

The image helper: src/lib/unsplash.js

`imageFor(photo)` returns the attributes an `<img>` needs:

- `src`: one Unsplash URL, cropped to the tile's shape (a tall tile asks for an image twice as high as it is wide).
- `srcSet`: the same photo at three widths, so the browser can pick a smaller file on a phone.
- `sizes`: tells the browser how wide the tile will be at each breakpoint (full width, half width from 40rem, a quarter or half from 64rem), matching the grid.

`PhotoTile` calls it once per photo and puts each result on its `<img>`.

Components and props

A component is a function that takes one object, its **props**, and returns JSX. Props are how a parent hands data down:

```
function Container({ className = "", children }) {
  return <div className={`mx-auto w-full max-w-7xl px-4 sm:px-6 lg:px-10
${className}`}>{children}</div>;
}
```

`children` is whatever sits between `<Container>` and `</Container>`. `className = ""` is a default value. The backtick string (a template literal) joins the shared classes with any extra ones the caller passes.

Small components exist so markup isn't repeated: `Container` wraps every section, `Eyebrow` draws the small label above a heading, `FormField` pairs a `<label>` with its input.

State 1: the category filter (Gallery.jsx)

```
const [activeCategory, setActiveCategory] = useState("all");

const visiblePhotos =
  activeCategory === "all"
    ? photos
    : photos.filter((photo) => photo.category === activeCategory);
```

1. `useState("all")` gives a value (`activeCategory`) and a function to change it (`setActiveCategory`). It starts as `"all"`.
2. Only the chosen category is stored. `visiblePhotos` is **derived**: worked out again on every render with `.filter()`, so it can never disagree with the buttons.
3. `Gallery` passes `active={activeCategory}` and `onChange={setActiveCategory}` to `FilterBar`.
4. `FilterBar` has no state. It marks the matching button with `aria-pressed` and dark styling, and calls `onChange(category.id)` when one is clicked.
5. Calling the setter makes React re-run `Gallery`, so the grid and "Showing n of 8" (`visiblePhotos.length` of `photos.length`) update together.

State 2: the shortlist and the controlled ShortlistToggle

`ShortlistToggle.jsx` is a **controlled component**: it shows what it is told and owns no state.

```
function ShortlistToggle({ value, onChange, label }) {
  return (
    <button aria-pressed={value} onClick={() => onChange(!value)} ...>
      {value ? "Shortlisted" : "Shortlist"}
    </button>
  );
}
```

- `value` is true or false: is this photo shortlisted?

- On click it doesn't change itself. It calls `onChange(!value)` to ask for the opposite, and waits for the parent to pass the new value back.
- `label` is the photo title, used in the screen-reader text ("Add Savanna at sunset to shortlist").

The real state lives at the top, in `App.jsx`:

```
const [shortlist, setShortlist] = useState([]);   // an array of photo ids

function setShortlisted(id, isShortlisted) {
  setShortlist((prev) =>
    isShortlisted ? [...prev, id] : prev.filter((x) => x !== id)
  );
}
```

- Adding copies the old array and puts the id on the end (`[...prev, id]`); removing filters it out. State is never changed in place, so React notices the new array.
- `App` passes `shortlist` and `setShortlisted` to `Gallery`. `Gallery` gives each `PhotoTile` `isShortlisted={shortlist.includes(photo.id)}`. `PhotoTile` hands that to its toggle as `value`, and turns `onChange(next)` into `onShortlistChange(photo.id, next)`.

Why the shortlist lives in `App` rather than in each tile ("lifting state up"):

- **It survives filtering.** A tile hidden by the filter is removed from the page; state kept inside it would be lost.
- **The header can count it.** `Header` gets `shortlistCount={shortlist.length}` and shows "Shortlist (n)".
- **The form can use it.** `App` turns the ids into titles (derived again, not stored) and passes them to `BookingForm`, which lists them, sends them in a hidden field and has a Clear shortlist button (`setShortlist([])`).

The booking form (BookingForm.jsx)

The form posts to Formspree using its normal `action` and `method`. One more piece of state, `status`, holds a message. While `FORM_ACTION` still contains `YOUR_FORM_ID`, `handleSubmit` calls `event.preventDefault()` to stop the submit and sets `status` to say nothing was sent. Once a real Formspree ID is in place, the handler does nothing and the browser submits normally.

Rendering lists with .map() and key

Every repeated element is made with `.map()`: photos, filter buttons, nav links, About facts, session types. Each item gets a `key` (such as `key={photo.id}`) so React can tell items apart when the list changes, for example after filtering.

Quick reference

To change	Open
Colours or fonts (light and dark)	`src/index.css`, the `@theme` block and the dark-mode block under it
Add, remove or reorder photos	`src/data/photos.js`, the `photos` array
Filter buttons	`categories` in `src/data/photos.js`
Image sizes or crops	`src/lib/unsplash.js`
Page title, favicon, Google Fonts	`index.html`
Nav links	`navLinks` in `src/components/Header.jsx`
Hero heading and intro	`src/components/Hero.jsx`
Grid columns and tile spans	`Gallery.jsx` (columns), `PhotoTile.jsx` (`shapeClasses`)
About text and facts	`src/components/About.jsx`
Form fields, Formspree ID, session types	`src/components/BookingForm.jsx`
Page width and side padding	`src/components/Container.jsx`